

ÉCOSYSTÈME

Cours complet interactif : Séance 1 – Écosystème Java (4 heures)



PAR ABOH ABOH YANNICK THIERRY



PROGRAMMATION

JAVA



INSTITUT AFRICAIN D'INFORMATIQUE

2026

Ot-11.0377908

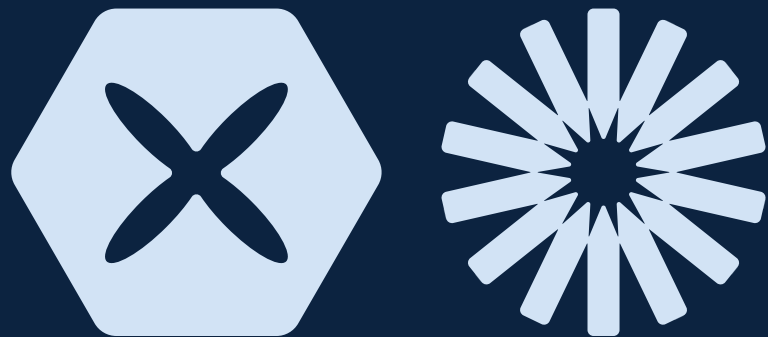
Filer Software & porce Collection
sáyat Probie vlay
Firetkastovplai.com



Pourquoi Java ?

Contexte historique de Java en bref

Java a été créé en 1991 sous le nom de projet “**Oak**” et a évolué au fil des ans, devenant l’un des langages de programmation les plus populaires grâce à ses caractéristiques polyvalentes et robustes.



Historique

Projet “Oak”



En 1991, James Gosling commence le développement de Java.

Renommé “Java”



En 1995, le langage est officiellement renommé Java.

JDK 1.0



En 1996, la première version du JDK est lancée.

Acquisition par Oracle



En 2010, Oracle acquiert la plateforme Java.

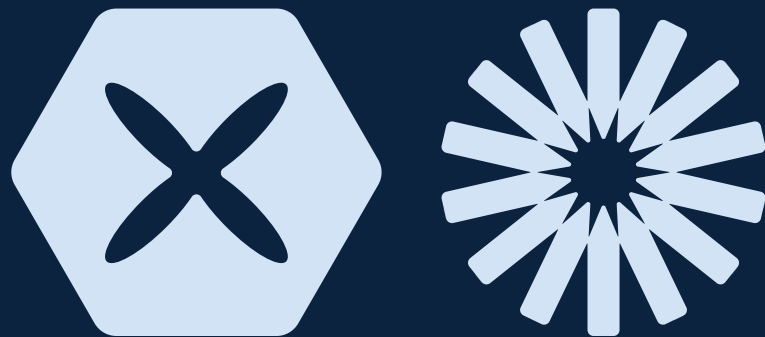
Questions interactives

Qui a déjà programmé en Java ?

Impliquer le public en demandant qui a déjà eu l'occasion de programmer en Java, et pour quels projets spécifiques. Cela permet de partager des expériences et d'engager la discussion.

Pourquoi Java reste-t-il populaire ?

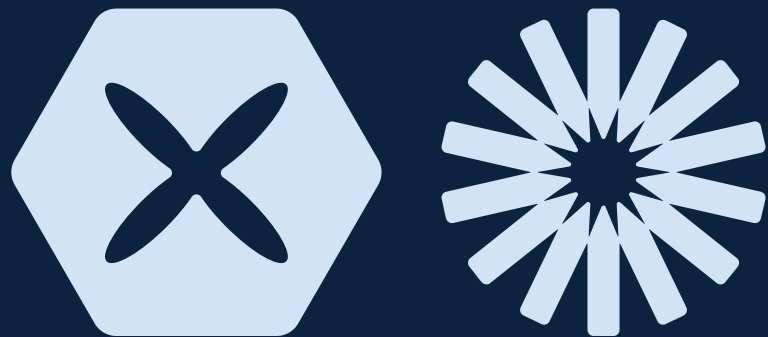
Discuter des raisons de la **longévité de Java**, y compris sa robustesse, sa communauté active et son utilisation dans des domaines variés, contribuant ainsi à sa pertinence continue dans le développement logiciel.



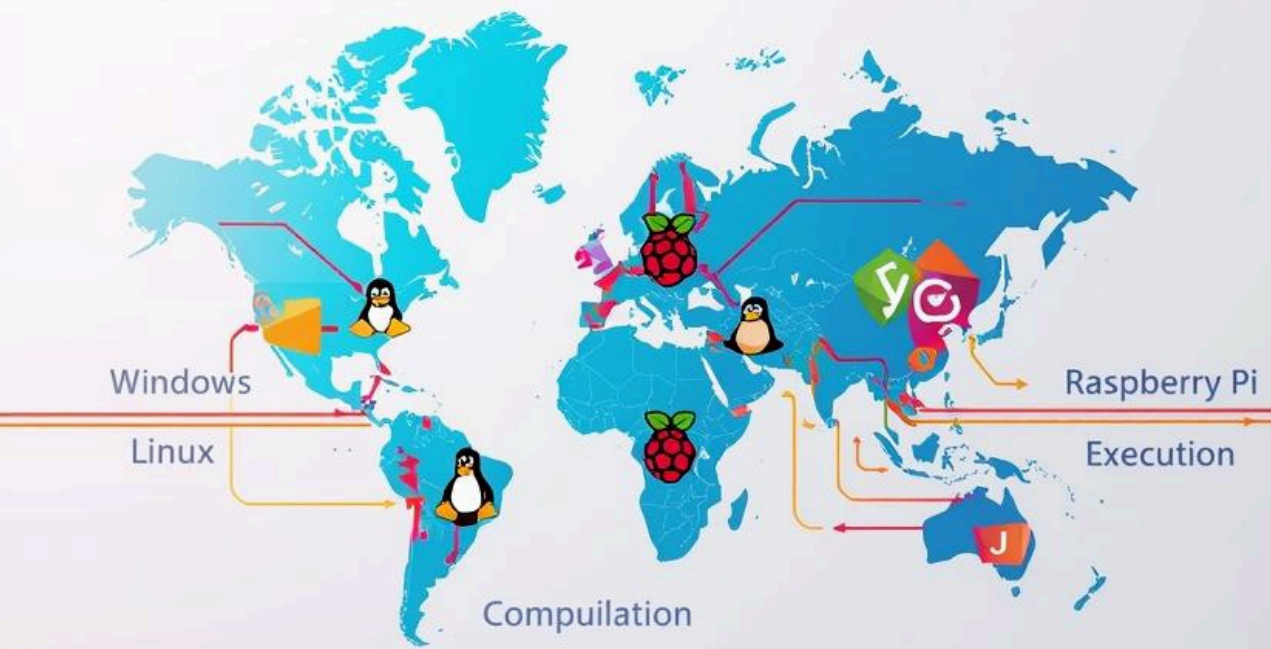
Philosophie WORA

Comprendre l'idée derrière Java

La philosophie WORA permet d'écrire du code une fois et de l'exécuter sur n'importe quelle plateforme. Par exemple, un programme Java compilé sous Windows peut être exécuté sur Linux.



WORA en Action



Compilation et exécution multiplateforme

Cette illustration montre comment une application Java peut être compilée sur Windows et exécutée sur Linux ou Raspberry Pi, soulignant l'approche WORA.



Applications Web

Java powers popular frameworks like Spring Boot and Jakarta EE, enabling robust web applications for diverse industries.

Mobile

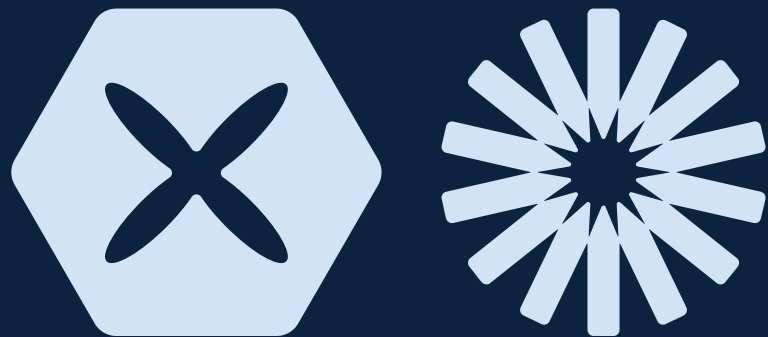
Java is essential for Android development, providing a strong foundation for mobile applications and innovative user experiences.

Domaines clés de Java

Introduction à la JVM

Imaginer les composants essentiels de la JVM

La Java Virtual Machine (JVM) est cruciale pour l'exécution des applications Java. **Imaginez** ses cinq composants principaux, chacun jouant un rôle essentiel dans le fonctionnement harmonieux des programmes.





Chargement

Le ClassLoader **charge les classes** nécessaires à l'exécution d'une application Java, tout comme un bibliothécaire organise les livres.

Liens

Il gère les relations entre les classes, en établissant des connexions et en assurant la disponibilité des ressources.

Étapes

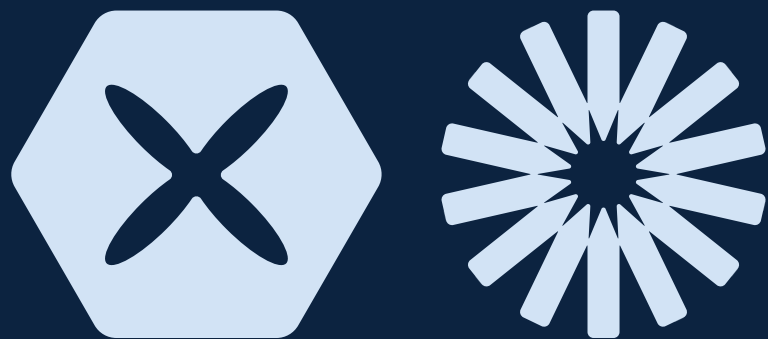
Le processus de chargement comprend trois étapes : chargement, liaison et initialisation, chacune essentielle pour le bon fonctionnement de l'application.

Composants principaux du ClassLoader

Zones mémoire JVM

Comprendre les différentes zones de mémoire

La JVM utilise plusieurs zones de mémoire pour gérer les données : **Method Area** pour les classes, **Heap** pour les objets, **Stack** pour les threads, **PC Register** pour la position d'exécution, et **Native Method Stack** pour le code natif.



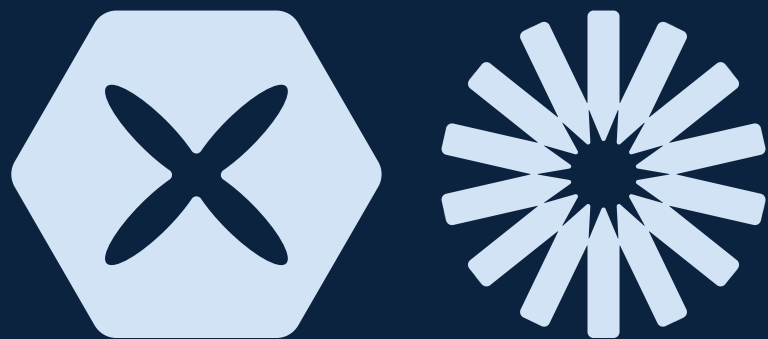
Questions sur la mémoire JVM

Modification d'un objet par plusieurs threads

Lorsque deux threads modifient le même objet dans le Heap, cela peut entraîner des **incohérences et des comportements imprévisibles** dans le programme. La synchronisation est essentielle pour éviter cela.

Importance de la Stack par thread

Chaque thread possède sa propre Stack pour gérer ses **appel de méthodes et variables locales**. Cela permet d'éviter les conflits d'accès et garantit l'isolement des données.



Code JVM Simple

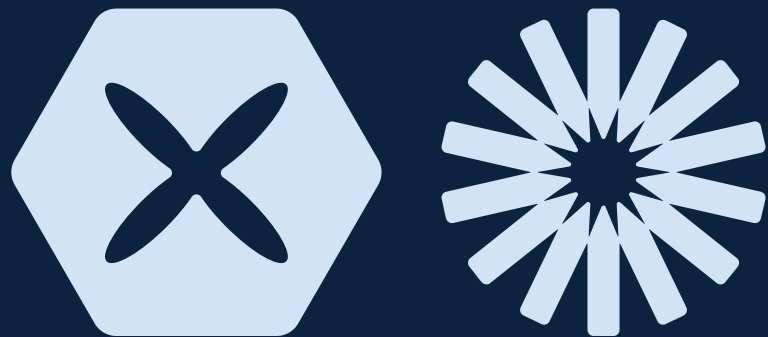
Zones mémoire en action

Cette illustration montre comment les variables sont allouées dans la mémoire en Java, mettant en évidence le Stack et le Heap.

Cycle de vie d'une application

Étapes de la compilation à l'exécution

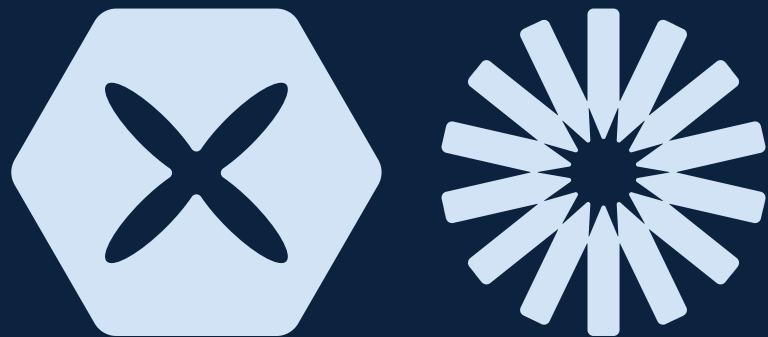
Le **cycle de vie** d'une application Java commence par l'écriture du code source, suivi de sa compilation en bytecode, et se termine par son exécution sur la machine virtuelle Java.



Bytecode expliqué

Comprendre le fonctionnement de la JVM

Le bytecode est comme un manuel IKEA, fournissant des instructions détaillées pour la JVM, permettant l'exécution d'applications Java sur diverses plateformes avec facilité et efficacité.



JDK, JRE, et JVM



Comprendre les différences par une analogie

Dans un restaurant, le **JDK** est le cuisinier avec outils, le **JRE** est le cuisinier avec ingrédients, et la **JVM** est le cuisinier suivant la recette.



JDK

Je suis **l'outil de développement** complet, contenant javac pour compiler et javadoc pour la documentation du code.

JRE

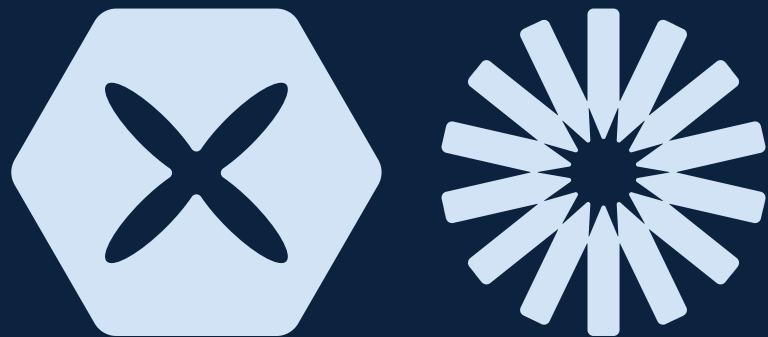
Je suis **l'environnement d'exécution**, permettant d'exécuter des fichiers .class sans compilation et d'assurer le bon fonctionnement des applications.

Qui suis-je ?

Comparatif JDK / JRE / JVM

Différences clés pour comprendre leur utilisation

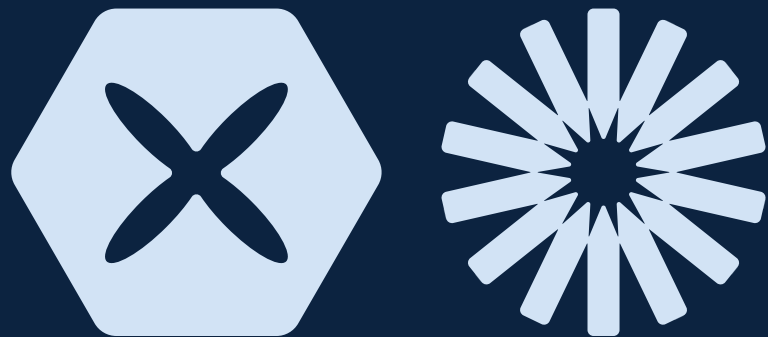
Ce tableau résume les différences essentielles entre JDK, JRE et JVM, en mettant en évidence leurs rôles, outils nécessaires et scénarios d'utilisation dans le développement et l'exécution d'applications Java.



Installation OpenJDK

Étapes clés pour une configuration réussie

Pour installer OpenJDK, il faut d'abord **télécharger le bon package**, puis l'extraire. Ensuite, configurez les variables d'environnement comme JAVA_HOME pour garantir le bon fonctionnement des applications Java.



Contact Information

Email

yannickabohthierry@gmail.com

Github

@yannickaboh

Telephone

(+241) 74 83 56 31

